Short tutorials from old www-page

Haavard Rue (hrue@r-inla.org)

KAUST, Aug 2020

Contents

- Contents
- Introduction
- User-defined priors for the hyperparameters
- Does INLA support the use of different link-functions?
- How can I do predictions using INLA?
- Some of the models needs a graph, how do I specify it?
- How `INLA` deal with `NA`
- Can INLA deal with missing covariates?
- Compute cross-validation or predictive measures of fit
- I have access to a remote Linux/MacOS server, is it possible to run the computations remotely and running R locally?
- Posteriors for linear combinations
- INLA seems to work great for near all cases, but are there cases where INLA is known to have problems?
- Can I have the linear predictor from one model as a covariate in a different model?
- Latent models, likelihoods and priors.
- Copying a model
- Replicate a model
- Models with more than one type of likelihood
- Models where the response/data depends on linear combinations of the "linear predictor" (or the sum of "fixed" and "random" effects)

Introduction

This vignette is a copy and slightly edited FAQ-entries and other short tutorials from the old `https://http://www.r-inla.org` page. The content comes is slightly random order, sorry about that. We run all examples here in classic-mode, for this reason.

```
inla.setOption(inla.mode="classic")
```

User-defined priors for the hyperparameters

If you want to use a prior for the hyperparameter that is not yet implemented there are two choices. If you think that your prior should be on the list and that other might use it to, please let us know. Alternatively, you can define your own prior using \verb|prior = "expression: ...."|, or by specifiying a table of x and y values which define the prior distribution.

There are three ways to specify prior distributions for hyperparameters in INLA:

- Use an available prior distribution
- Define your own prior distribution function using R-like (not equal) syntax as expression.
- Create a table of (x, y) values which represent your prior distribution.

In the following we will provide more details regarding the two last options. Finally, we will present an example illustrating (and comparing) the three different possibilities by means of the log-gamma distribution for the precision parameter.

A user can specify any (univariate) prior distribution for the hyperparameter θ by defining an expression for the log-density $\log \pi(\theta)$, as a function of the corresponding θ. It is important to be aware that θ is on the internal scale.

The format is

```
expression: statement; statement; ...; return(value)
```

where "statement" is any regular statement (more below) and "value" is the value for the log-density of the prior, evaluated at the current value for θ.

Here, is an example defining the log-gamma distribution:

```
prior.expression = "expression:
    a = 1;
    b = 0.1;
    precision = exp(log_precision);
    logdens = log(b^a) - lgamma(a)
       + (a-1)*log_precision - b*precision;
    log_jacobian = log_precision;
    return(logdens + log_jacobian);"
```

Some syntax specific notes: * No white-space before "(.)" in the return statement. * A ";" is needed to terminate each expression. * A "_" is allowed in variable names.

Known functions that can be used within the expression statement are

- common math functions, such as exp, sin, ...
- "gamma" denotes the gamma-function and "lgamma" is its log
- `x^y` is expressed as either `x^y` or `pow(x,y)`

Instead of defining a prior distribution function, it is possible to provide a table of suitable values x (internal scale) and the corresponding log-density values y. INLA fits a spline through the provided points and continues with this in the succeeding computations. Note, there is no transformation into a functional form performed or required. The input-format for the table is a string, which starts with `table:` and is then followed by a block of x-values and a block of the corresponding y-values, which represent the values of the log-density evaluated on x. Thus

```
table: x_1 ... x_n y_1 ... y_n
```

We illustrate all three different ways of defining a prior distribution for the precision of a normal likelihood. To show that the three definitions lead to the same result we inspect the logmarginal likelihood.

```
## the loggamma-prior
prior.function = function(log_precision) {
    a = 1;
    b = 0.1;
    precision = exp(log_precision);
    logdens = log(b^a) - lgamma(a) + (a-1)*log_precision - b*precision;
    log_jacobian = log_precision;
    return(logdens + log_jacobian)
}

## implementing the loggamma-prior using "expression:"
prior.expression = "expression:
a = 1;
b = 0.1;
precision = exp(log_precision);
logdens = log(b^a) - lgamma(a)
    + (a-1)*log_precision - b*precision;
log_jacobian = log_precision;
return(logdens + log_jacobian);"

## use suitable support points x
lprec = seq(-10, 10, len=100)
## link the x and corresponding y values into a
## string which begins with "table:""
prior.table = paste(c("table:", cbind(lprec,
                    prior.function(lprec))), collapse=" ", sep="")

# simulate some data
n = 50
y = rnorm(n)

## use the built-in loggamma prior
r1 = inla(y~1,data = data.frame(y),
control.family = list(hyper = list(prec = list(
                      prior = "loggamma", param = c(1, 0.1)))))

## use the definition using expression
r2 = inla(y~1, data = data.frame(y),
          control.family = list(hyper = list(
                                prec = list(prior = prior.expression))))

## use a table of x and y values representing the loggamma prior
r3 = inla(y~1, data = data.frame(y),
          control.family = list(hyper = list(
                                prec = list(prior = prior.table))))

print(round(c(r1$mlik[1], r2$mlik[1], r3$mlik[1]), digits=3))
```

```
## [1] -70.7 -70.7 -70.7
```

Does INLA support the use of different link-functions?

Yes, the type of link function is given in the `control.family` statement using `control.link=...`, and the type of link-functions implemented are listed on the documentation for each likelihood. The default link is `default` which corresponds to the second link function in the list. Here is an example

```
n = 100
z = rnorm(n)
eta = 1 + 0.1*z
N = 2

p = inla.link.invlogit(eta)
y = rbinom(n,  size = N, prob = p)
r = inla(y ~ 1 + z,  data = data.frame(y, z), family = "binomial", Ntrials = rep(N, n),
        control.family = list(control.link = list(model="logit")),
        control.predictor = list(compute=TRUE))

p = inla.link.invprobit(eta)
y = rbinom(n,  size = N, prob = p)
rr = inla(y ~ 1 + z,  data = data.frame(y, z), family = "binomial", Ntrials = rep(N, n),
        control.family = list(control.link = list(model="probit")),
        control.predictor = list(compute=TRUE))

p = inla.link.invcloglog(eta)
y = rbinom(n,  size = N, prob = p)
rrr = inla(y ~ 1 + z,  data = data.frame(y, z), family = "binomial", Ntrials = rep(N, n),
        control.family = list(control.link = list(model="cloglog")),
        control.predictor = list(compute=TRUE))
```

Other linkfunctions/models are also avilable from within `R`, see `?inla.link`

How can I do predictions using INLA?

In `INLA` there is no function `predict` as for `glm/lm` in `R`. Predictions must to done as a part of the model fitting itself. As prediction is the same as fitting a model with some missing data, we can simply set `y[i] = NA` for those "locations" we want to predict. Here is a simple example

```
n = 100
n.pred = 10
y = arima.sim(n=n, model=list(ar=0.9))
N = n + n.pred
yy = c(y, rep(NA, n.pred))
i = 1:N
formula = yy ~ f(i, model="ar1")
r = inla(formula, data = data.frame(i,yy),
        control.family = list(initial = 10, fixed=TRUE)) ## no observational noise
```

which gives predictions

```
r$summary.random$i[(n+1):N, c("mean", "sd") ]
```

```
##     mean   sd
```

```
## 101 2.29 1.56
## 102 2.09 1.74
## 103 1.92 1.88
## 104 1.76 1.98
## 105 1.62 2.07
## 106 1.49 2.13
## 107 1.37 2.19
## 108 1.27 2.24
## 109 1.18 2.27
## 110 1.09 2.31
```

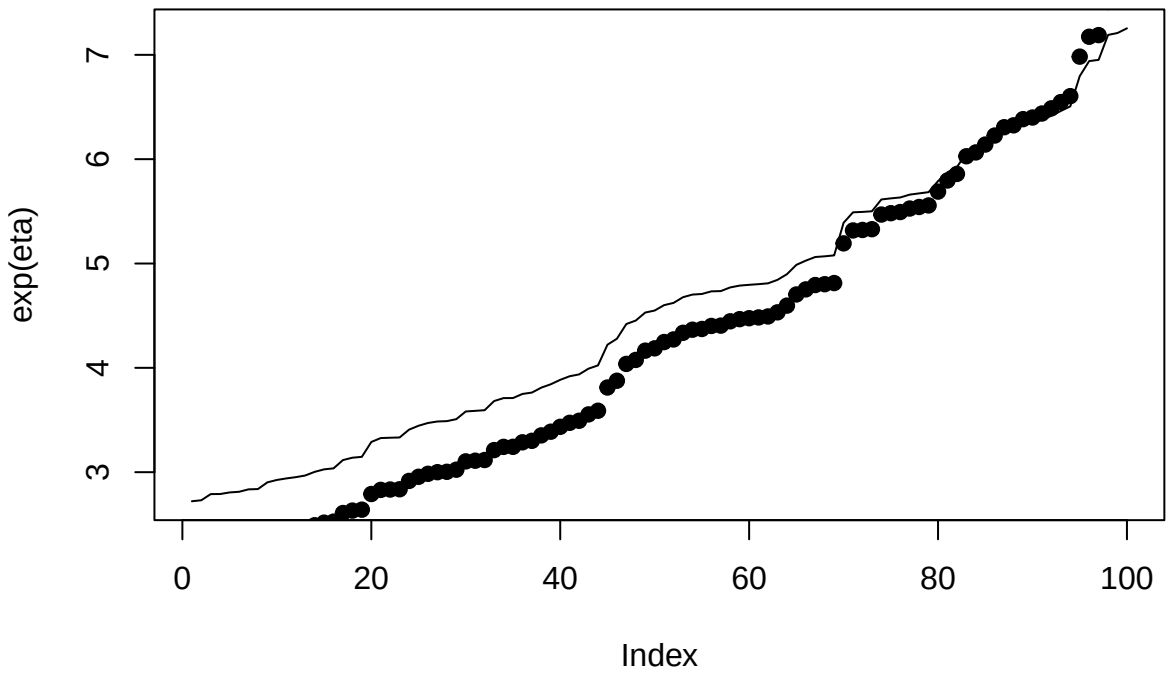
Quantiles such like `r$summary.fitted.values` and `r$marginals.fitted.values`, if computed, use the identity link if `y[i] = NA` by default. If you want the `fitted.values` computed with a different link function, then there are two ways to doit.

In the case you want to use the link-function from the likelihood already used (most often the case), there is the argument `link` in `control.predictor`. If the response `y[idx] = NA`, then set `link[idx] = 1`, to indicate that you want to compute that fitted value using the link function from `family[1]`. With several likelihoods, set `link[idx]` to family-index which is correct, ie the column number in the response. The following example shows the usage:

```
## simple poisson regression
n = 100
x = sort(runif(n))
eta = 1 + x
lambda = exp(eta)
y = rpois(n, lambda = lambda)

## missing values:
y[1:3] = NA
y[(n-2):n] = NA

## link = 1 is a shortcut for rep(1, n) where n is the appropriate
## length. here '1' is a reference to the first 'family', ie
## 'family[1]'
r = inla(y ~ 1 + x,  family = "poisson",
        data = data.frame(y, x),
        control.predictor = list(link = 1))
plot(exp(eta),type ="l")
points(r$summary.fitted.values$mean, pch=19)
```

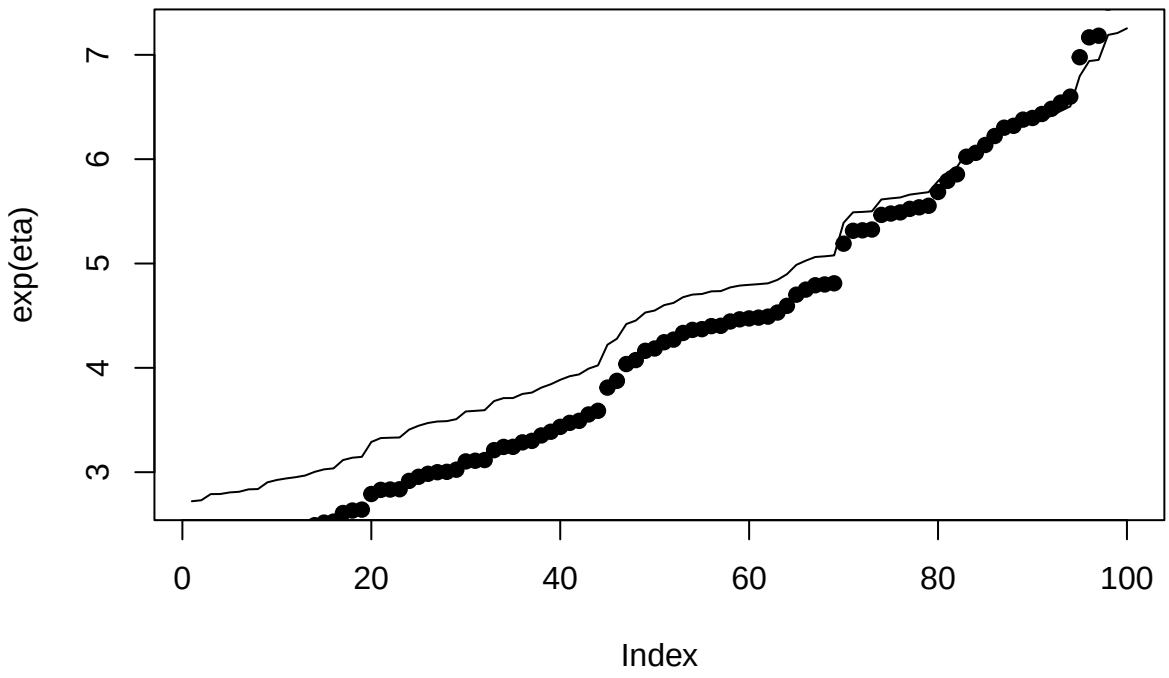


We only need to define `link` where there are missing values. Entries for which the observation is not `NA`, is ignored.

For more than one likelihood, use '2' to refer to the second likelihood. Here is an example where we split the data in two, and assign the second half the `nbinomial` distribution.

```
n2 = n %/% 2L
Y = matrix(NA, n, 2)
Y[1:n2, 1] = y[1:n2]
Y[1:n2 + n2, 2] = y[1:n2 + n2]
link = rep(NA, n)
link[which(is.na(y[1:n2]))] = 1
link[n2  + which(is.na(y[1:n2 + n2]))] = 2

r = inla(Y ~ 1 + x,  family = c("poisson", "nbinomial"),
        data = list(Y=Y, x=x),
        control.family= list(list(), list()),
        control.predictor = list(link = link))
plot(exp(eta),type ="l")
points(r$summary.fitted.values$mean, pch=19)
```



We can transform marginals manually using the function `inla.marginal.transform` or compute expectations using `inla.emarginal`, like in this example (taken from `demo(Tokyo)`).

```
## Load the data
data(Tokyo)
summary(Tokyo)
```

```
##        y                n          time
##  Min.   :0.000   Min.   :1   Min.   :  1.0
##  1st Qu.:0.000   1st Qu.:2   1st Qu.: 92.2
##  Median :0.000   Median :2   Median :183.5
##  Mean   :0.525   Mean   :2   Mean   :183.5
##  3rd Qu.:1.000   3rd Qu.:2   3rd Qu.:274.8
##  Max.   :2.000   Max.   :2   Max.   :366.0
```

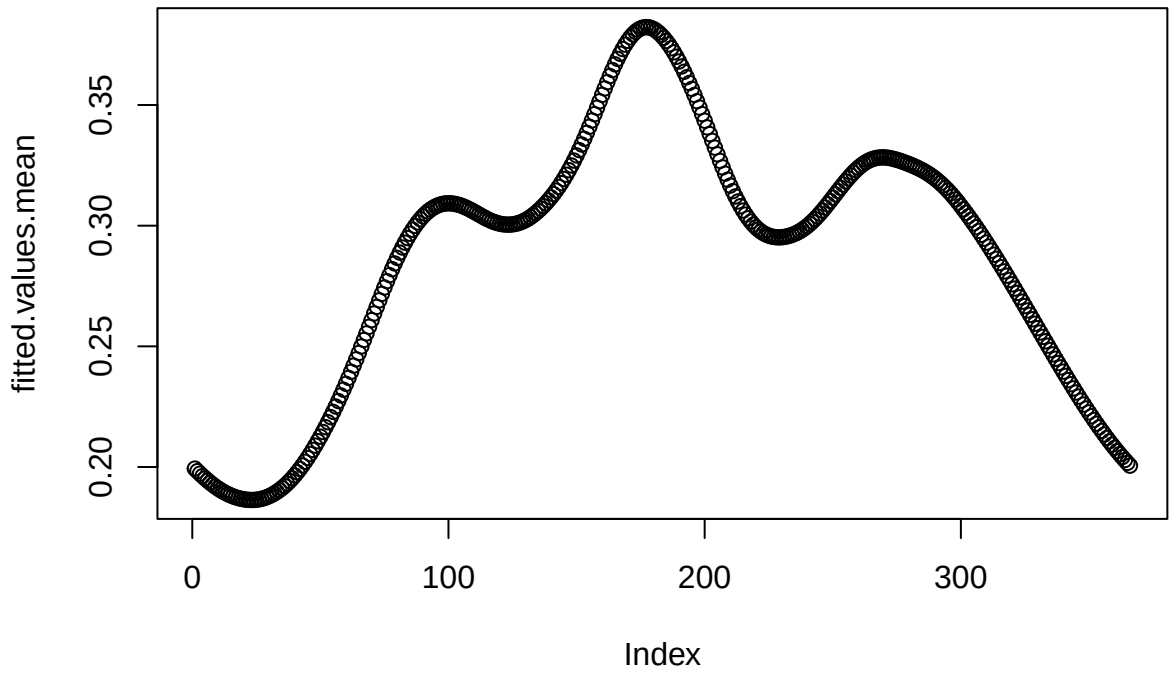
```
Tokyo$y[300:366] <- NA

## Define the model
formula = y ~ f(time, model="rw2", scale.model=TRUE,
                constr=FALSE, cyclic=TRUE,
                hyper = list(prec=list(prior="pc.prec",
                                       param=c(2,0.01)))) -1

## We'll get a warning since we have not defined the link argument
result = inla(formula, family="binomial", Ntrials=n, data=Tokyo,
    control.compute = list(return.marginals.predictor = TRUE),
```

```
    control.predictor=list(compute=T))

## need to recompute the fitted values for those with data[i] = NA,
## as the identity link is used.
n = 366
fitted.values.mean = numeric(n)
for(i in 1:366) {
    if (is.na(Tokyo$y[i])) {
        if (FALSE) {
            ## either like this, which is slower
            marg = inla.marginal.transform(
                            function(x) exp(x)/(1+exp(x)),
                            result$marginals.fitted.values[[i]] )
            fitted.values.mean[i] = inla.emarginal(function(x) x, marg)
        } else {
            ## or like this,  which is faster
            fitted.values.mean[i] = inla.emarginal(
                            function(x) exp(x)/(1 +exp(x)),
                            result$marginals.fitted.values[[i]])
        }
    } else {
        fitted.values.mean[i] = result$summary.fitted.values[i,"mean"]
    }
}
plot(fitted.values.mean)
```



Some of the models needs a graph, how do I specify it?

Some of the models in `INLA` needs the user to specify a graph, saying which nodes are neighbours to each other. A 'graph' can be specified in three different ways.

- As an ascii or binary file with a graph specification, or the same contents given as (possible list of) mix of character and numerics arguments.
- As a symmetric (dense or sparse) matrix, where the non-zero pattern of the matrix defines the graph.
- As an inla.graph-object

A graph defined in an ascii-file, must have the following format. The first entry is the number of nodes in the graph, n. The nodes in the graph are labelled $1, 2, \ldots, n$. The next entries, specify the number of neighbours and the neighbours for each node. A simple example is the following

```
4
1 2 3 4
2 0
3 1 1
4 1 1
```

This defines a graph with four nodes, where node 1 has 2 neighbours 3 and 4, node 2 as 0 neighbours, node 3 has 1 neighbour 1, and node 4 has 1 neighbour 1, and the graph looks like this (requires `Rgraphviz` and `graph`)

```
g = inla.read.graph("4 1 2 3 4 2 0 3 1 1 4 1 1")
if (FALSE && requireNamespace("Rgraphviz") && requireNamespace("graph")) {
  tryCatch(
    plot(g),
    error = function(e) {
      message("Rgraphviz plotting doesn't seem to work on this installation.")
    }
  )
}
```

Note that we need to specify the implied symmetry as well. In this example 4 is a neighbour of 1, then we also need to specify that 1 is a neighbour of 4.

Instead of storing the graph specification on a file, it can also be specified as a character string with the same contents as a file, like

```
"4 1 2 3 4  2 0 3 1 1 4 1 1"
```

as used in `inla.read.graph` above.

Due to imitations of the length of a string/line, so in practice, this way specifying the graph, seems more useful for teaching or demonstration purposes than for practical analysis.

Within `INLA`, this would look like

```
formula = y ~ f(idx, model = "besag", graph = "graph.dat")
```

or

```
formula = y ~ f(idx, model = "besag", graph = "4 1 2 3 4  2 0 3 1 1 4 1 1")
```

A graph can also be defined as a symmetric (dense or sparse) matrix, where the non-zero pattern of the matrix defines the graph A neighbour matrix is often used for defining which nodes that are neighbours, with the convention that if `Q[i,j] != 0` then i and j are neighbours if $i \neq j$.

For example, the (dense) matrix C

```
C = matrix(c(1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1),4,4)
C
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    0    1    1
## [2,]    0    1    0    0
## [3,]    1    0    1    0
## [4,]    1    0    0    1
```

defines the same graph show above.

Since graphs tends to be large, we often define them as a sparse matrix

```
C.sparse= inla.as.sparse(C)
C.sparse
```

```
## 4 x 4 sparse Matrix of class "dgTMatrix"
##
## [1,] 1 . 1 1
## [2,] . 1 . .
## [3,] 1 . 1 .
## [4,] 1 . . 1
```

We can then use `graph=C` or `graph=C.sparse` in the formula.

We can also define the graph as an `inla.graph`-object, which is used internally, represent a graph. For example

```
str(g)
```

```
## List of 4
##  $ n   : int 4
##  $ nnbs: num [1:4] 2 0 1 1
##  $ nbs :List of 4
##   ..$ : int [1:2] 3 4
##   ..$ : num(0)
##   ..$ : int 1
##   ..$ : int 1
##  $ cc  :List of 4
##   ..$ id   : int [1:4] 1 2 1 1
##   ..$ n    : int 2
##   ..$ nodes:List of 2
##   .. ..$ : int [1:3] 1 3 4
##   .. ..$ : int 2
##   ..$ mean : Factor w/ 1 level "1": 1 NA 1 1
##  - attr(*, "class")= chr "inla.graph"
```

and use `graph = g` as the argument.

The internal format, are as follows. `n` is the size of the graph. `nnbs` are the number of neighbours to each node, `nbs` list all the neighbours to each node, and the class is `inla.graph`. The `cc`-list is for internal use only and specify the connected components in the graph.

`INLA` has some functions to work with graphs, and here is a short summary.

- `inla.read.graph()` and `inla.write.graph()`, read and write graphs using any of the graph specifications above.
- You can plot a `inla.graph`-object using `plot()` and get a summary using `summary()`. The plotting requires the `Rgraphviz` package.
- From a graph specification, you can generate the neighbour matrix, using `inla.graph2matrix()`. You can plot a graph specification as a neighbour matrix, using `inla.spy()`
- If you have 'errors' in your graph, you may read it using `inla.debug.graph()`. This is only available for a graph specification in an ascii-file.

How `INLA` deal with `NA`

For a formula like

```
formula = y ~ x + f(k, model= <some model>)
```

then `NA`'s in either y, x or k are treated differently.

`NA`'s in the response y. If `y[i] = NA`, this means that `y[i]` is not observed, hence gives no contribution to the likelihood.

`NA`'s in fixed effect x. If `x[i] = NA` this means that `x[i]` is not part of the linear predictor for `y[i]`. For fixed effects, this is equivalent to `x[i]=0`, hence internally we make this change: `x[is.na(x)] = 0`.

`NA`'s in random effect k. If `k[i] = NA`, this means that the random effect does not contribute to the linear predictor for `y[i]`.

`NA`'s in a factor x. `NA`'s in a factor x is not allowed unless `NA` is a level in itself, or

```
control.fixed = list(expand.factor.strategy = "inla")
```

is set. With this option, then `NA` is interpreted similarly as a fixed effect, where `NA` means no contribution from x. The effect of `expand.factor.strategy="inla"`, is best explained with an example.

```
r = inla(y ~ 1 + x, data = data.frame(y=1:3, x=factor(c("a","b","c"))))
as.matrix(r$model.matrix)
```

```
##   (Intercept) xb xc
## 1           1  0  0
## 2           1  1  0
## 3           1  0  1
```

for default value of the argument `contrasts`. The effect of `xa` is removed to make the corresponding matrix non-singular. If we want to expand `x` into each of each three effects, then we can do

```
r = inla(y ~ 1 + x, data = data.frame(y=1:3,x=factor(c("a","b","c"))),
         control.fixed = list(expand.factor.strategy="inla"))
as.matrix(r$model.matrix)
```

```
##   (Intercept) xa xb xc
## 1           1  1  0  0
## 2           1  0  1  0
## 3           1  0  0  1
```

As we see, each level of the factor is now treated symetrically. Although the corresponding frequentist matrix is singular as we have confounding with the intercept, the Bayesian posterior is still proper with proper priors.

With a `NA` in `x`, we get

```
r = inla(y ~ 1 + x, data = data.frame(y=1:3,x=factor(c("a","b",NA))),
         control.fixed = list(expand.factor.strategy="inla"))
as.matrix(r$model.matrix)
```

```
##   (Intercept) xa xb
## 1           1  1  0
## 2           1  0  1
## 3           1  0  0
```

so that the 3rd element of the linear predictor has no contribution from `x`, as it should.

Can INLA deal with missing covariates?

No, INLA has no generic way to "impute" or integrate-out missing covariates. You have to adjust your model to account for missing covariates, like using one of the measurement error models ("meb", "mec"), or construct a joint model for the data and the covariates, but this is case-specific.

Compute cross-validation or predictive measures of fit

`INLA` provides two types of leave-one-out predictive measures of fit. It is the CPO value, which is

$$\text{Prob}(y_i|y_{-i}),$$

the PIT value

$$\text{Prob}(y_i^{\text{new}} \leq y_i|y_{-i})$$

To enable the computation of these quantities, you will need to add the argument

```
control.compute=list(cpo=TRUE)
```

We can also compute PO values

$$\text{Prob}(y_i|y),$$

when argument `po=TRUE` is added.

If the resulting object is `result`, then you will find the predictive quantities as `result$cpo$cpo` and `result$cpo$pit`.

Implicit assumptions made in for computations, and there are internal checks that these are satisfied. The results of these checks will appear as `result$cpo$failure`. In short, if `result$cpo$failure[i] > 0` then some assumption is violated, the higher the value (maximum 1) the more seriously. If `result$cpo$failure[i] == 0` then the assumptions should be ok.

You *may* want to recompute those with non-zero failure. However, this must be done *manually* by removing `y[i]` from the dataset, fit the model and then predict `y[i]`. To provide a more efficient implementation of this, we have provided

```
improved.result = inla.cpo(result)
```

which take an inla-object which is the output from `inla()`, and recompute (in an efficient way) the cpo/pit for which `result$cpo$failure > 0`, and return 'result' with the improved estimates of cpo/pit. See `?inla.cpo` for details.

I have access to a remote Linux/MacOS server, is it possible to run the computations remotely and running R locally?

Yes! This option allow `INLA` to use a remote server to do the computations. In order to use this feature, you need to do some setup which is different from (various) Linux distributions, Mac and Windows. In short:

- install `R` and `R-INLA` a remote server, for example `foo.bar.org`.
- Install your public ssh-key on `foo.bar.org` to setup password free access to the remove server using `ssh`. And please check that this is indeed working before moving forward!
- On your local host, run `inla.remote()` to initialise the init-file `~/.inlarc` and then edit this file to fit your needs.
- You may now have to log out and log in again, to make sure your ssh key is signed out.
- You can now use option `inla.call="remote"` to do the computations on your remote server, or set this globally with `inla.setOption("inla.call", "remote")`

You can also submit a job on the remote server, so you do not need to sit and wait for it to finish, but you can collect the results later. Basically, you do

```
r = inla(..., inla.call = "submit")
```

which will start the job on the server. You can start many jobs, and list them using

```
inla.qstat()
```

and you can fetch the results (for the job above) using

```
r = inla.qget(r)
```

You can also delete jobs and fetch the jobs from another machine; see `?inla.q` for further details.

HOWTO setup ssh-keys: For the unexperienced user, this is somewhat tricky; sorry about that. The easiest is to find a friend that knows this and can help you. Newer system do a lot of these things very nicely these days.

It is also possible to setup this from Windows using CYGWIN, and INLA can work with this interface as well. Please see the old web-page for details, which are long and technical. HOWEVER, I am no longer convinced that this work anymore, as I haven't seen this is use for years. It is much much easier to use a virtual machine with Linux on Windows.

Posteriors for linear combinations

I have some linear combinations of the nodes in the latent field that I want to compute the posterior marginal of, is that possible? Yes! These are called 'linear combinations'. There are handy functions, 'inla.make.lincomb()' and 'inla.make.lincombs()', to define one or many such linear combinations. Single linear combinations made by using 'inla.make.lincomb()' can easily be joined into many. Its use is easiest explained using a rather long example...

Here is the example, that explains these features.

```
## A simple model
n = 100
a = rnorm(n)
b = rnorm(n)
idx = 1:n

y = rnorm(n) + a + b
formula = y ~ 1 + a + b + f(idx, model="iid")

## assume we want to compute the posterior for
##
##  2 * beta_a + 3 * beta_b + idx[1] - idx[2]
##
## which we specify as follows (and giving it a unique name)

lc1 = inla.make.lincomb(a=2, b=3, idx = c(1,-1,rep(NA,n-2)))
names(lc1) = "lc1"

## strictly speaking, it is sufficient to use `idx = c(1,-1)', as the
## remaining idx's are not used in any case.

r = inla(formula, data = data.frame(a,b,y),
        ## add the linear combinations here
        lincomb = lc1,
        ## force noise variance to be essiatially zero
        control.family = list(initial=10, fixed=TRUE))

## to verify the result, we can compare the mean but the variance and
## marginal cannot be computed from the simpler marginals alone.
lc1.1 = 2 * r$summary.fixed["a", "mean"] + 3 * r$summary.fixed["b",
    "mean"] + r$summary.random$idx$mean[1] -
    r$summary.random$idx$mean[2]
lc1.2= r$summary.lincomb.derived$mean
print(round(c(lc1.1 = lc1.1, lc1.2 = lc1.2), dig=3))
```

```
## lc1.1 lc1.2
##  3.57  3.57
```

The marginals are available as `r$marginals.lincomb.derived$...`

There is an another function which is handy for specifying many linear combinations at once, that is `inla.make.lincombs()` (note the plural s). Here each 'row' define one linear combination

```
## let wa and wb be vectors, and we want to compute the marginals for
## beta_a * wa[i] + beta_b * wb[i], for i=1..m. this is done
## conveniently as follows

m = 10
wa = runif(m)
wb = runif(m)
lc.many = inla.make.lincombs(a = wa, b=wb)

## we can give them names as well, but there are also default names, like
print(names(lc.many))
```

```
##  [1] "lc01" "lc02" "lc03" "lc04" "lc05" "lc06" "lc07" "lc08" "lc09"
## [10] "lc10"
```

```
r = inla(formula, data = data.frame(a,b,y),
        lincomb = lc.many,
        control.family = list(initial=10, fixed=TRUE))
print(round(r$summary.lincomb.derived, dig=3))
```

```
##      ID  mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## lc01  1 1.314 0.118      1.082    1.314      1.547 1.314   0
## lc02  2 1.664 0.127      1.414    1.664      1.914 1.664   0
## lc03  3 0.726 0.081      0.567    0.726      0.885 0.726   0
## lc04  4 0.424 0.029      0.367    0.424      0.482 0.424   0
## lc05  5 1.751 0.130      1.496    1.751      2.007 1.751   0
## lc06  6 1.862 0.136      1.595    1.862      2.128 1.862   0
## lc07  7 1.124 0.085      0.957    1.124      1.291 1.124   0
## lc08  8 0.168 0.012      0.145    0.168      0.191 0.168   0
## lc09  9 0.729 0.071      0.590    0.729      0.869 0.729   0
## lc10 10 1.541 0.109      1.325    1.541      1.756 1.541   0
```

Terms like 'idx' above, can be added as `idx = IDX` into `inla.make.lincombs()`, where IDX is a matrix. Again, each row of the arguments define one linear combination.

There is a further option available for the derived linear combinations, that is the option to compute also the posterior correlation matrix between all the linear combinations. To activate this option, use

```
control.inla = list(lincomb.derived.correlation.matrix = TRUE)
```

and you will find the resulting posterior correlation matrix as

```
result$misc$lincomb.derived.correlation.matrix
```

Here is a small example where we compute the correlation matrix for the predicted values of a hidden AR(1) model with an intercept.

```
n = 100
nPred = 10
phi = 0.9
x = arima.sim(n,  model = list(ar=phi)) * sqrt(1-phi^2)
```

```
y = 1 + x + rnorm(n,  sd=0.1)

time = 1:(n + nPred)
Y = c(y, rep(NA, nPred))
formula = Y ~ 1 + f(time, model="ar1")

## make linear combinations which are the nPred linear predictors
B = matrix(NA, nPred, n+nPred)
for(i in 1:nPred) {
    B[i, n+i] = 1
}
lcs = inla.make.lincombs(Predictor = B)

r = inla(formula,  data = data.frame(Y, time),
        control.predictor = list(compute=TRUE),
        lincomb = lcs,
        inla.mode="classic",
        control.inla = list(lincomb.derived.correlation.matrix=TRUE))

print(round(r$misc$lincomb.derived.correlation.matrix,dig=3))
```

```
##       lc01  lc02  lc03  lc04  lc05  lc06  lc07  lc08  lc09  lc10
## lc01 1.000 0.676 0.527 0.437 0.373 0.326 0.289 0.259 0.234 0.213
## lc02 0.676 1.000 0.780 0.645 0.551 0.481 0.426 0.381 0.344 0.313
## lc03 0.527 0.780 1.000 0.827 0.706 0.616 0.545 0.487 0.440 0.400
## lc04 0.437 0.645 0.827 1.000 0.853 0.743 0.657 0.587 0.530 0.481
## lc05 0.373 0.551 0.706 0.853 1.000 0.870 0.769 0.687 0.618 0.561
## lc06 0.326 0.481 0.616 0.743 0.870 1.000 0.882 0.787 0.708 0.642
## lc07 0.289 0.426 0.545 0.657 0.769 0.882 1.000 0.891 0.801 0.725
## lc08 0.259 0.381 0.487 0.587 0.687 0.787 0.891 1.000 0.898 0.812
## lc09 0.234 0.344 0.440 0.530 0.618 0.708 0.801 0.898 1.000 0.903
## lc10 0.213 0.313 0.400 0.481 0.561 0.642 0.725 0.812 0.903 1.000
```

INLA seems to work great for near all cases, but are there cases where INLA is known to have problems?

The methodology needs the full conditional density for the latent field to be "near" Gaussian. This is usually achived by either replications or smoothing/"borrowing strength". A simple example which do not have this, is the following:

```
n = 100
u = rnorm(n)
eta = 1 + u
p = exp(eta)/(1+exp(eta))
y = rbinom(n, size=1,  prob = p)

idx = 1:n
result = inla(y ~ 1 + f(idx, model="iid",
                        hyper = list(prec = list(prior="pc.prec",
                                                 prior = c(1,0.01)))),
              data =data.frame(y,idx), family = "binomial",
```

```
              Ntrials = 1)
summary(result)
```

```
## Time used:
##     Pre = 0.0895, Running = 0.102, Post = 0.00676, Total = 0.199
## Fixed effects:
##              mean    sd 0.025quant 0.5quant 0.975quant mode kld
## (Intercept) 0.492 0.207      0.092     0.49      0.903 0.49   0
##
## Random effects:
##   Name     Model
##     idx IID model
##
## Model hyperparameters:
##                         mean         sd 0.025quant 0.5quant 0.975quant
## Precision for idx 88109.17 1526302.41       8.95   211.31  144270.97
##                      mode
## Precision for idx 13.36
##
## Marginal log-Likelihood:  -67.15
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

For each binary observation there is an iid "random effect" u, and there is no smoothing/"borrowing strength'' (apart from the weak intercept). If you plot the loglikelihood for eta for $y = 1$, say, then its an increasing function for increasing eta, so the likelihood itself would like $\eta = \infty$. With an unknown precision for u we run into problems; INLA has a tendency to estimate a to high precision for u. However, it must be noted that the model is almost singular and you'll have a strong prior sensitivity in the (exact) results as well. There is a similar discussion in here as well for the Salamander data example.

Can I have the linear predictor from one model as a covariate in a different model?

Yes, this is possible. Essentially, you have to set the linear predictor for the first model equal to 'u', and then you can copy 'u' and use the scaling to get the regression coefficient. A simple example will illustrate the idea:

```
## simple example
set.seed(12345L)
sd.data = 1
n = 2000

## fixed high and low precisions are specified with parameters
fixed.high <- 10
fixed.low <- - 6

## otherwise is the model identical to the previous FAQ-model
x1 = rnorm(n)
eta1 = 1 + x1
x2 = rnorm(n)
eta2 = 2 + 2*eta1 + 2*x2
```

```
y1 = rnorm(n, mean=eta1, sd = sd.data)
y2 = rnorm(n, mean=eta2, sd = sd.data)
## the trick is to create a vector 'u' (iid) which is equal to eta1, and then we can copy 'u' to
## create beta*u or beta*eta1. we do this by using 0 = eta1 -u + tiny.noise

formula = Y ~ -1 + intercept1 + X1 + intercept2 +
    f(u, w, model="iid", hyper = list(prec = list(initial = fixed.low, fixed=TRUE))) +
    f(b.eta2, copy="u", hyper = list(beta = list(fixed = FALSE))) + X2
Y = matrix(NA, 3*n, 3)

## part 1: y1
intercept1 = rep(1, n)
X1 = x1
intercept2 = rep(NA, n)
u = rep(NA, n)
w = rep(NA, n)
b.eta2 = rep(NA, n)
X2 = rep(NA, n)
Y[1:n, 1] = y1
## part 2: 0 = eta1 - u + tiny.noise
intercept1 = c(intercept1,  intercept1)
X1 = c(X1,  x1)
intercept2 = c(intercept2,  rep(NA, n))
u = c(u, 1:n)
w = c(w, rep(-1, n))
b.eta2 = c(b.eta2,  rep(NA, n))
X2 = c(X2,  rep(NA, n))
Y[n + 1:n, 2] = 0
## part 3: y2
intercept1 = c(intercept1,  rep(NA, n))
X1 = c(X1,  rep(NA, n))
intercept2 = c(intercept2, rep(1, n))
u = c(u, rep(NA, n))
w = c(w, rep(NA, n))
b.eta2 = c(b.eta2, 1:n)
X2 = c(X2, x2)
Y[2*n + 1:n, 3] = y2
r = inla(formula,
    data =  list(Y=Y, intercept1=intercept1, X1=X1,
        intercept2=intercept2, u=u, w=w, b.eta2=b.eta2, X2=X2),
    family = c("gaussian", "gaussian", "gaussian"),
    control.family = list(
        list(),
        list(hyper = list(prec = list(initial = fixed.high, fixed=TRUE))),
        list()))
summary(r)
```

```
## Time used:
##     Pre = 0.121, Running = 0.929, Post = 0.0387, Total = 1.09
## Fixed effects:
##             mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## intercept1 0.977 0.023      0.932    0.977       1.02 0.977   0
## X1         1.008 0.022      0.966    1.008       1.05 1.008   0
```

```
## intercept2 2.054 0.070      1.915    2.055       2.19 2.055   0
## X2         2.031 0.024      1.984    2.031       2.08 2.031   0
##
## Random effects:
##   Name     Model
##     u IID model
##    b.eta2 Copy
##
## Model hyperparameters:
##                                           mean    sd 0.025quant
## Precision for the Gaussian observations    0.969 0.031       0.91
## Precision for the Gaussian observations[3] 0.969 0.031       0.91
## Beta for b.eta2                            1.994 0.049       1.90
##                                           0.5quant 0.975quant  mode
## Precision for the Gaussian observations       0.969       1.03 0.969
## Precision for the Gaussian observations[3]    0.969       1.03 0.968
## Beta for b.eta2                               1.994       2.09 1.992
##
## Marginal log-Likelihood:  -13642.62
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

```r
## An alternative specification of the same model applies the inla.stack() function.
## It is called four times, one time for each submodel, one time for the "copying",
## and one time for joining the three substacks to one.

## Part 1:
## The first model stack comes in two versions

mod1a.stack <- inla.stack(
    data = list(Y = as.matrix(cbind(y1, NA, NA))),
    A = list(1, 1),
    effects = list(intercept1 = rep(1,n),
            x1 = x1)
)

mod1b.stack <- inla.stack(
    data = list(Y = as.matrix(cbind(y1, NA, NA))),
    A = list(1),
    effects = list(u = 1:n)
)

## Part 2:
copy.stack <- inla.stack(
    data = list(Y = as.matrix(cbind(NA, rep(0,n), NA))),
    A = list(1, 1, - 1),
    effects = list(intercept1 = rep(1,n),
            x1 = x1,
            u = 1:n)
)

## Part 3:
mod2.stack <- inla.stack(
```

```
    data = list(Y = as.matrix(cbind(NA, NA, y2))),
    A = list(1, 1, 1),
    effects = list(intercept2 = rep(1,n),
            x2 = x2,
            b.eta2 = 1:n)
)

all.a.stack <- inla.stack(mod1a.stack, copy.stack, mod2.stack)
all.b.stack <- inla.stack(mod1b.stack, copy.stack, mod2.stack)

## The formula is slightly modified because the long vectors X1 and X2 has not been created
## and there is no weight applied as "-1" appears as the effect of u

formula = Y ~ -1 + intercept1 + x1 + intercept2 + x2 +
    f(u, model = "iid", hyper = list(prec = list(initial = fixed.low, fixed = TRUE))) +
    f(b.eta2, copy = "u", hyper = list(beta = list(fixed = FALSE)))

## The inla() function makes use of the stack
r.new.a = inla(formula,
    data =  inla.stack.data(all.a.stack),
    control.predictor = list(A = inla.stack.A(all.a.stack)),
    family = c("gaussian", "gaussian", "gaussian"),
    control.family = list(
        list(),
        list(hyper = list(prec = list(initial = fixed.high, fixed=TRUE))),
        list()))
summary(r.new.a)
```

```
## Time used:
##     Pre = 0.124, Running = 1.47, Post = 0.0515, Total = 1.65
## Fixed effects:
##             mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## intercept1 0.977 0.023      0.932    0.977       1.02 0.977   0
## x1         1.008 0.022      0.966    1.008       1.05 1.008   0
## intercept2 2.054 0.070      1.915    2.055       2.19 2.055   0
## x2         2.031 0.024      1.984    2.031       2.08 2.031   0
##
## Random effects:
##   Name     Model
##     u IID model
##    b.eta2 Copy
##
## Model hyperparameters:
##                                           mean    sd 0.025quant
## Precision for the Gaussian observations    0.970 0.031      0.911
## Precision for the Gaussian observations[3] 0.969 0.031      0.910
## Beta for b.eta2                            1.994 0.049      1.900
##                                           0.5quant 0.975quant  mode
## Precision for the Gaussian observations       0.969       1.03 0.969
## Precision for the Gaussian observations[3]    0.969       1.03 0.968
## Beta for b.eta2                               1.994       2.09 1.992
##
## Marginal log-Likelihood:  -13642.62
```

```
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

```
r.new.b = inla(formula,
    data =  inla.stack.data(all.b.stack),
    control.predictor = list(A = inla.stack.A(all.b.stack)),
    family = c("gaussian", "gaussian", "gaussian"),
    control.family = list(
        list(),
        list(hyper = list(prec = list(initial = fixed.high, fixed=TRUE))),
        list()))
summary(r.new.b)
```

```
## Time used:
##     Pre = 0.132, Running = 1.37, Post = 0.048, Total = 1.55
## Fixed effects:
##             mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## intercept1 0.977 0.023      0.932    0.977       1.02 0.977   0
## x1         1.008 0.022      0.966    1.008       1.05 1.008   0
## intercept2 2.054 0.070      1.915    2.055       2.19 2.055   0
## x2         2.031 0.024      1.984    2.031       2.08 2.031   0
##
## Random effects:
##   Name     Model
##     u IID model
##    b.eta2 Copy
##
## Model hyperparameters:
##                                          mean    sd 0.025quant
## Precision for the Gaussian observations    0.970 0.031       0.91
## Precision for the Gaussian observations[3] 0.969 0.031       0.91
## Beta for b.eta2                            1.994 0.049       1.90
##                                          0.5quant 0.975quant  mode
## Precision for the Gaussian observations       0.969       1.03 0.969
## Precision for the Gaussian observations[3]    0.969       1.03 0.968
## Beta for b.eta2                               1.994       2.09 1.992
##
## Marginal log-Likelihood:  -13642.60
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

```
## All specifications give similar and acceptable results both for fixed effects and precisions,
## The version r.new.b seems faster than r.new.a for more complex models with one or more spde-terms
```

Latent models, likelihoods and priors.

The list of latent models, likelihood and priors implemented, can be found by doing (or give a specific section, see `?inla.list.models`)

```
inla.list.models()
```

```
## Section [group]
##      ar                          AR(p) correlations
##      ar1                         AR(1) correlations
##      besag                       Besag model
##      exchangeable                Exchangeable correlations
##      exchangeablepos             Exchangeable positive correlations
##      iid                         Independent model
##      rw1                         Random walk of order 1
##      rw2                         Random walk of order 2
##  Section [hazard]
##      iid                         An iid model for the log-hazard
##      rw1                         A random walk of order 1 for the log-hazard
##      rw2                         A random walk of order 2 for the log-hazard
##  Section [latent]
##      2diid                       (This model is obsolute)
##      ar                          Auto-regressive model of order p (AR(p))
##      ar1                         Auto-regressive model of order 1 (AR(1))
##      ar1c                        Auto-regressive model of order 1 w/covariates
##      besag                       The Besag area model (CAR-model)
##      besag2                      The shared Besag model
##      besagproper                 A proper version of the Besag model
##      besagproper2                An alternative proper version of the Besag model
##      bym                         The BYM-model (Besag-York-Mollier model)
##      bym2                        The BYM-model with the PC priors
##      cgeneric                    Generic latent model specified using C
##      clinear                     Constrained linear effect
##      copy                        Create a copy of a model component
##      crw2                        Exact solution to the random walk of order 2
##      dmatern                     Dense Matern field
##      fgn                         Fractional Gaussian noise model
##      fgn2                        Fractional Gaussian noise model (alt 2)
##      generic                     A generic model
##      generic0                    A generic model (type 0)
##      generic1                    A generic model (type 1)
##      generic2                    A generic model (type 2)
##      generic3                    A generic model (type 3)
##      iid                         Gaussian random effects in dim=1
##      iid1d                       Gaussian random effect in dim=1 with Wishart prior
##      iid2d                       Gaussian random effect in dim=2 with Wishart prior
##      iid3d                       Gaussian random effect in dim=3 with Wishart prior
##      iid4d                       Gaussian random effect in dim=4 with Wishart prior
##      iid5d                       Gaussian random effect in dim=5 with Wishart prior
##      iidkd                       Gaussian random effect in dim=k with Wishart prior
##      intslope                    Intecept-slope model with Wishart-prior
##      linear                      Alternative interface to an fixed effect
##      log1exp                     A nonlinear model of a covariate
##      logdist                     A nonlinear model of a covariate
##      matern2d                    Matern covariance function on a regular grid
##      meb                         Berkson measurement error model
##      mec                         Classical measurement error model
##      ou                          The Ornstein-Uhlenbeck process
```

```
##      prw2                        Proper random walk of order 2
##      revsigm                     Reverse sigmoidal effect of a covariate
##      rgeneric                    Generic latent model specified using R
##      rw1                         Random walk of order 1
##      rw2                         Random walk of order 2
##      rw2d                        Thin-plate spline model
##      rw2diid                     Thin-plate spline with iid noise
##      scopy                       Create a scopy of a model component
##      seasonal                    Seasonal model for time series
##      sigm                        Sigmoidal effect of a covariate
##      slm                         Spatial lag model
##      spde                        A SPDE model
##      spde2                       A SPDE2 model
##      spde3                       A SPDE3 model
##      z                           The z-model in a classical mixed model formulation
##  Section [likelihood]
##      0binomial                   New 0-inflated Binomial
##      0binomialS                  New 0-inflated Binomial Swap
##      0nbinomial                  New 0-inflated nBinomial
##      0nbinomialS                 New 0-inflated nBinomialS
##      0poisson                    New 0-inflated Poisson
##      0poissonS                   New 0-inflated Poisson Swap
##      1poisson                    New 1-inflated Poisson
##      1poissonS                   New 1-inflated Poisson Swap
##      agaussian                   The aggregated Gaussian likelihoood
##      bcgaussian                  The Box-Cox Gaussian likelihoood
##      bell                        The Bell likelihood
##      beta                        The Beta likelihood
##      betabinomial                The Beta-Binomial likelihood
##      betabinomialna              The Beta-Binomial Normal approximation likelihood
##      bgev                        The blended Generalized Extreme Value likelihood
##      binomial                    The Binomial likelihood
##      binomialmix                 Binomial mixture
##      cbinomial                   The clustered Binomial likelihood
##      cennbinomial2               The CenNegBinomial2 likelihood (similar to cenpoisson2)
##      cenpoisson                  Then censored Poisson likelihood
##      cenpoisson2                 Then censored Poisson likelihood (version 2)
##      circularnormal              The circular Gaussian likelihoood
##      cloglike                    User-defined likelihood
##      coxph                       Cox-proportional hazard likelihood
##      dgompertzsurv               destructive gompertz (survival) distribution
##      dgp                         Discrete generalized Pareto likelihood
##      egp                         Exteneded Generalized Pareto likelihood
##      exponential                 The Exponential likelihood
##      exponentialsurv             The Exponential likelihood (survival)
##      exppower                    The exponential power likelihoood
##      fl                          The fl likelihood
##      fmri                        fmri distribution (special nc-chi)
##      fmrisurv                    fmri distribution (special nc-chi)
##      gamma                       The Gamma likelihood
##      gammacount                  A Gamma generalisation of the Poisson likelihood
##      gammacountmean              Another Gamma generalisation of the Poisson likelihood
##      gammajw                     A special case of the Gamma likelihood
##      gammajwsurv                 A special case of the Gamma likelihood (survival)
```

```
##      gammasurv                The Gamma likelihood (survival)
##      gammasv                  The Gamma likelihood with constant rate
##      gaussian                 The Gaussian likelihoood
##      gaussianjw               The GaussianJW likelihoood
##      gev                      The Generalized Extreme Value likelihood
##      ggaussian                Generalized Gaussian
##      ggaussianS               Generalized GaussianS
##      gompertz                 gompertz distribution
##      gompertzsurv             gompertz distribution
##      gp                       Generalized Pareto likelihood
##      gpoisson                 The generalized Poisson likelihood
##      iidgamma                 (experimental)
##      iidlogitbeta             (experimental)
##      lavm                     Link adjusted von Mises circular distribution
##      loggammafrailty          (experimental)
##      logistic                 The Logistic likelihoood
##      loglogistic              The loglogistic likelihood
##      loglogisticsurv          The loglogistic likelihood (survival)
##      lognormal                The log-Normal likelihood
##      lognormalsurv            The log-Normal likelihood (survival)
##      logperiodogram           Likelihood for the log-periodogram
##      mgamma                   The modal Gamma likelihood
##      mgammasurv               The modal Gamma likelihood (survival)
##      nbinomial                The negBinomial likelihood
##      nbinomial2               The negBinomial2 likelihood
##      nmix                     Binomial-Poisson mixture
##      nmixnb                   NegBinomial-Poisson mixture
##      npoisson                 The Normal approximation to the Poisson likelihood
##      nvm                      Normal approx of the von Mises circular distribution
##      nzpoisson                The nzPoisson likelihood
##      obeta                    The ordered Beta likelihood
##      occupancy                Occupancy likelihood
##      poisson                  The Poisson likelihood
##      poisson.special1         The Poisson.special1 likelihood
##      pom                      Likelihood for the proportional odds model
##      qkumar                   A quantile version of the Kumar likelihood
##      qloglogistic             A quantile loglogistic likelihood
##      qloglogisticsurv         A quantile loglogistic likelihood (survival)
##      rcpoisson                Randomly censored Poisson
##      sem                      The SEM likelihoood
##      simplex                  The simplex likelihood
##      sn                       The Skew-Normal likelihoood
##      stdgaussian              The stdGaussian likelihoood
##      stochvol                 The Gaussian stochvol likelihood
##      stochvolln               The Log-Normal stochvol likelihood
##      stochvolnig              The Normal inverse Gaussian stochvol likelihood
##      stochvolsn               The SkewNormal stochvol likelihood
##      stochvolt                The Student-t stochvol likelihood
##      t                        Student-t likelihood
##      tpoisson                 Thinned Poisson
##      tstrata                  A stratified version of the Student-t likelihood
##      tweedie                  Tweedie distribution
##      vm                       von Mises circular distribution
##      weibull                  The Weibull likelihood
```

```
##      weibullsurv                    The Weibull likelihood (survival)
##      wrappedcauchy                  The wrapped Cauchy likelihoood
##      xbinomial                      The Binomial likelihood (experimental version)
##      xpoisson                       The Poisson likelihood (expert version)
##      zeroinflatedbetabinomial0      Zero-inflated Beta-Binomial, type 0
##      zeroinflatedbetabinomial1      Zero-inflated Beta-Binomial, type 1
##      zeroinflatedbetabinomial2      Zero inflated Beta-Binomial, type 2
##      zeroinflatedbinomial0          Zero-inflated Binomial, type 0
##      zeroinflatedbinomial1          Zero-inflated Binomial, type 1
##      zeroinflatedbinomial2          Zero-inflated Binomial, type 2
##      zeroinflatedcenpoisson0        Zero-inflated censored Poisson, type 0
##      zeroinflatedcenpoisson1        Zero-inflated censored Poisson, type 1
##      zeroinflatednbinomial0         Zero inflated negBinomial, type 0
##      zeroinflatednbinomial1         Zero inflated negBinomial, type 1
##      zeroinflatednbinomial1strata2 Zero inflated negBinomial, type 1, strata 2
##      zeroinflatednbinomial1strata3 Zero inflated negBinomial, type 1, strata 3
##      zeroinflatednbinomial2         Zero inflated negBinomial, type 2
##      zeroinflatedpoisson0           Zero-inflated Poisson, type 0
##      zeroinflatedpoisson1           Zero-inflated Poisson, type 1
##      zeroinflatedpoisson2           Zero-inflated Poisson, type 2
##      zeroninflatedbinomial2         Zero and N inflated binomial, type 2
##      zeroninflatedbinomial3         Zero and N inflated binomial, type 3
##  Section [link]
##      cauchit                        The cauchit-link
##      ccloglog                       The complement complementary log-log link
##      cgevit                         Complement GEVIT link
##      cloglog                        The complementary log-log link
##      default                        The default link
##      gevit                          GEVIT link
##      identity                       The identity link
##      inverse                        The inverse link
##      log                            The log-link
##      loga                           The loga-link
##      logit                          The logit-link
##      logitoffset                    Logit-link with an offset
##      loglog                         The log-log link
##      logoffset                      Log-link with an offset
##      neglog                         The negative log-link
##      powerlogit                     Power logit link
##      pquantile                      The population quantile-link
##      probit                         The probit-link
##      quantile                       The quantile-link
##      robit                          Robit link
##      sn                             Skew-normal link
##      special1                       A special1-link function (experimental)
##      special2                       A special2-link function (experimental)
##      sslogit                        Logit link with sensitivity and specificity
##      tan                            The tan-link
##      tanpi                          The tanpi-link
##      test1                          A test1-link function (experimental)
##  Section [lp.scale]
##  NA Section [mix]
##      gaussian                       Gaussian mixture
##      loggamma                       LogGamma mixture
```

```
##      mloggamma                    Minus-LogGamma mixture
##  Section [predictor]
##      predictor                    (do not use)
##  Section [prior]
##      betacorrelation              Beta prior for the correlation
##      dirichlet                    Dirichlet prior
##      expression:                  A generic prior defined using expressions
##      flat                         A constant prior
##      gamma                        Gamma prior
##      gaussian                     Gaussian prior
##      invalid                      Void prior
##      jeffreystdf                  Jeffreys prior for the doc
##      laplace                      Laplace prior
##      linksnintercept              Skew-normal-link intercept-prior
##      logflat                      A constant prior for log(theta)
##      loggamma                     Log-Gamma prior
##      logiflat                     A constant prior for log(1/theta)
##      logitbeta                    Logit prior for a probability
##      logtgaussian                 Truncated Gaussian prior
##      logtnormal                   Truncated Normal prior
##      minuslogsqrtruncnormal       (obsolete)
##      mvnorm                       A multivariate Normal prior
##      none                         No prior
##      normal                       Normal prior
##      pc                           Generic PC prior
##      pc.alphaw                    PC prior for alpha in Weibull
##      pc.ar                        PC prior for the AR(p) model
##      pc.cor0                      PC prior correlation, basemodel cor=0
##      pc.cor1                      PC prior correlation, basemodel cor=1
##      pc.dof                       PC prior for log(dof-2)
##      pc.egptail                   PC prior for the tail in the EGP likelihood
##      pc.fgnh                      PC prior for the Hurst parameter in FGN
##      pc.gamma                     PC prior for a Gamma parameter
##      pc.gammacount                PC prior for the GammaCount likelihood
##      pc.gevtail                   PC prior for the tail in the GEV likelihood
##      pc.matern                    PC prior for the Matern SPDE
##      pc.mgamma                    PC prior for a Gamma parameter
##      pc.prec                      PC prior for log(precision)
##      pc.prw2.range                PCprior for the range in PRW2
##      pc.range                     PC prior for the range in the Matern SPDE
##      pc.sn                        PC prior for the skew-normal
##      pc.spde.GA                   (experimental)
##      pom                          #classes-dependent prior for the POM model
##      ref.ar                       Reference prior for the AR(p) model, p<=3
##      rprior:                      A R-function defining the prior
##      table:                       A generic tabulated prior
##      wishart1d                    Wishart prior dim=1
##      wishart2d                    Wishart prior dim=2
##      wishart3d                    Wishart prior dim=3
##      wishart4d                    Wishart prior dim=4
##      wishart5d                    Wishart prior dim=5
##      wishartkd                    Wishart prior
##  Section [scopy]
##      rw1                          Random walk of order 1
```

```
##      rw2                             Random walk of order 2
##  Section [wrapper]
##      joint                           (experimental)
```

Copying a model

We often encounter the situation where an element of a model is needed more than once for each observation. One example is where

```
y = a + b*w + ...
```

for fixed weights w and where (a_i, b_i) is bivariate Normal and all 2-vectors are independent.

Using the model

```
f(idx, model="iid2d", n=2*m, ...)
```

provide a random vector v, say, with length $2m$ stored as

$$v = (a_1, a_2, ..., a_m, b_1, b_2, ..., b_m).$$

To implement this, we simply create an indentical copy of v, v^*, where $v == v^*$ (nearly). Using the copy-feature, we can do

```
idx = 1:m
idx.copy = m + 1:m
formula = y ~  f(idx, model="iid2d", n=2*m) + f(idx.copy, w, copy="idx") + ....
```

recalling that the first m elements is a and the last m elements are b, and where w are the weights.

The second `f()` terms define itself as a copy of `f(idx, ...)`, and it inherit some of its features, like `n` and `values`.

A copied model may also have an unknown scaling (hyperparameter), which is default fixed to be 1. In the following example, we will use this feature to estimate the unknown scaling (in this case, scaling is 2), assuming we know the precision for z.

```
n=1000
i=1:n
j = i
z = rnorm(n)
w = runif(n)
y = z  + 2*z*w + rnorm(n)
formula = y ~ f(i, model="iid",initial=0, fixed=T) +
              f(j, w, copy="i", fixed=FALSE)
r = inla(formula, data = data.frame(i,j,w,y))
summary(r)
```

```
## Time used:
##     Pre = 0.0975, Running = 0.294, Post = 0.0219, Total = 0.413
## Fixed effects:
##               mean    sd 0.025quant 0.5quant 0.975quant   mode kld
## (Intercept) -0.156 0.064     -0.281   -0.156      -0.03 -0.156   0
```

```
##
## Random effects:
##   Name     Model
##     i IID model
##    j Copy
##
## Model hyperparameters:
##                                       mean    sd 0.025quant
## Precision for the Gaussian observations 1.19 0.292      0.738
## Beta for j                              1.92 0.122      1.683
##                                       0.5quant 0.975quant mode
## Precision for the Gaussian observations     1.15       1.88 1.06
## Beta for j                                  1.91       2.16 1.90
##
## Marginal log-Likelihood:  -2207.33
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

If the scaling parameter is within given range, then option \verb|range = c(low, high)|, can be given. In this case

```
beta = low + (high-low)*exp(beta.local)/(1+exp(beta.local))
```

and the prior is defined on `beta.local`.

If `low=high` or `range = NULL`, then the identity mapping is used. If `high=Inf` and |verb|low!=Inf|, then the mapping `low + exp(beta.local)` is used. The case low=Inf and high!=Inf is not yet implemented.

A model or a copied model can be copied several times. The degree of closeness of v and v^* is specified by the argument `precision`, as the precision of the noise added to v to get v^*.

Replicate a model

Independent replications of a model with the same hyperparmeters can be defined using the argument `replicate`,

```
f(idx, model = ..,  replicate = r)
```

Here, `r` is a vector of the same length as `idx`. In this case, we use a two-dimensional index to index this (sub-)model: `(idx, r)`, so `(1,2)` identify the first value of the second replication of this model (component). Number of replications are defined as `max(replicate)`, unless it is defined by the argument `nrep`.

One example is the model 'iid':

```
i = 1:n
formula = y ~  f(i, model = "iid") + ...
```

which has an alternative equivalent formulation as 'n' replications of an iid-model with length 1

```
i = rep(1,n)
r = 1:n
formula = y ~  f(i, model="iid", replicate = r) + ...
```

In the following example, we estimate the parameters in three AR(1) time-series with the same hyperparameters (ie its replicated) but with separate means:

```
n = 100
y1 = arima.sim(n=n, model=list(ar=c(0.9)))+10
y2 = arima.sim(n=n, model=list(ar=c(0.9)))+20
y3 = arima.sim(n=n, model=list(ar=c(0.9)))+30

formula = y ~ mean -1 + f(i, model="ar1", replicate=r)
y = c(y1,y2,y3)
i = rep(1:n, 3)
r = rep(1:3, each=n)
mean = as.factor(r)
result = inla(formula, family = "gaussian",
              data = data.frame(y, i, mean),
              control.family = list(initial = 12, fixed=TRUE))
summary(result)
```

```
## Time used:
##     Pre = 0.0902, Running = 0.17, Post = 0.0101, Total = 0.27
## Fixed effects:
##        mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## mean1  9.85 0.757       8.35     9.85       11.4  9.85   0
## mean2 20.43 0.757      18.92    20.44       21.9 20.44   0
## mean3 29.18 0.757      27.69    29.17       30.7 29.17   0
##
## Random effects:
##   Name     Model
##     i AR1 model
##
## Model hyperparameters:
##                   mean    sd 0.025quant 0.5quant 0.975quant  mode
## Precision for i 0.261 0.057      0.159    0.258      0.380 0.256
## Rho for i       0.871 0.028      0.814    0.872      0.921 0.872
##
## Marginal log-Likelihood:  -443.12
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

All other arguments is interpreted for the basic model and also replicated. Like argument `constr=TRUE`, is interpreted as each replication sums to zero, and additional constraints are also replicated.

Models with more than one type of likelihood

There is no constraint in INLA that the type of likelihood must be the same for all observations. In fact, every observation could have its own likelihood. Extentions include more than one familily, like the Normal, Poisson, etc, but also having in the model groups of observations with separate hyperparameters within each group, where the family, for example, can be the same.

In the formula

```
y ~ a + 1
```

we allow `y` to be a matrix. In this case each column of `y` define one likelihood where the family is the same the hyperparameters are the same. For each row, only one of the columns could (but don't have to) have an observation (non-`NA` value), the other colums must have value `NA`. All other parameters to the likelihood, like `E` `Ntrials`, `offset` and `scale` are used as appropriate. Example: If row i column j is a Poission observation, then `E[i]` is used as the scaling. Similar with the others. This works as only one column for each row is non-`NA`.

The argument `family` is in the case where `y` is a matrix, a list of families. The argument `control.family` is then a list of lists; one for each family.

The first example, is a simple linear regression, where the first half of the data is observed with unknown precision `tau.1` (with a 'default' noninformative prior) and the second half of the data is observed with unknown precision `tau.2`. Otherwise, the two models have the same form for the linear predictor.

```
##  Simple linear regression with observations with two different
##  variances.
n = 100
N = 2*n
y = numeric(N)
x = rnorm(N)

y[1:n] = 1 + x[1:n] + rnorm(n, sd = 1/sqrt(1))
y[1:n + n] = 1 + x[1:n + n] + rnorm(n, sd = 1/sqrt(2))

Y = matrix(NA, N, 2)
Y[1:n, 1] = y[1:n]
Y[1:n + n, 2] = y[1:n + n]

formula = Y ~ x + 1
result = inla(
        formula,
        data = list(Y=Y, x=x),
        family = c("gaussian", "gaussian"),
        control.family = list(list(prior = "flat", param = numeric()),
                              list()))
summary(result)
```

```
## Time used:
##     Pre = 0.0837, Running = 0.117, Post = 0.0071, Total = 0.208
## Fixed effects:
##             mean    sd 0.025quant 0.5quant 0.975quant mode kld
## (Intercept) 1.07 0.058      0.959     1.07       1.19 1.07   0
## x           1.07 0.057      0.961     1.07       1.19 1.07   0
##
## Model hyperparameters:
##                                           mean    sd 0.025quant
## Precision for the Gaussian observations    1.06 0.151      0.791
## Precision for the Gaussian observations[2] 2.06 0.293      1.536
##                                           0.5quant 0.975quant mode
## Precision for the Gaussian observations         1.05       1.38 1.03
## Precision for the Gaussian observations[2]      2.04       2.69 2.00
##
```

```
## Marginal log-Likelihood:  -265.04
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

The second example shows how to use information from two sources to estimate the effect of the covariate x.

```
## Simple example with two types of likelihoods
n = 10
N = 2*n

## common covariates
x = rnorm(n)

## Poisson, depends on x
E1 = runif(n)
y1 = rpois(n, lambda = E1*exp(x))

## Binomial, depends on x
size = sample(1:10, size=n, replace=TRUE)
prob = exp(x)/(1+exp(x))
y2 = rbinom(n, size= size, prob = prob)

## Join them together
Y = matrix(NA, N, 2)
Y[1:n, 1] = y1
Y[1:n + n, 2] = y2

## The E for the Poisson
E = numeric(N)
E[1:n] = E1
E[1:n + n] = NA

## Ntrials for the Binomial
Ntrials = numeric(N)
Ntrials[1:n] = NA
Ntrials[1:n + n] = size

## Duplicate the covariate which is shared
X = numeric(N)
X[1:n] = x
X[1:n + n] = x

## Formula involving Y as a matrix
formula = Y ~ X - 1

## `family' is now
result = inla(formula,
        family = c("poisson", "binomial"),
        control.family=list(list(), list()),
        data = list(Y=Y, X=X),
        E = E, Ntrials = Ntrials)
summary(result)
```

```
## Time used:
##     Pre = 0.083, Running = 0.0992, Post = 0.00399, Total = 0.186
## Fixed effects:
##   mean    sd 0.025quant 0.5quant 0.975quant mode kld
## X 1.12 0.159      0.792     1.13       1.42   NA   0
##
## Marginal log-Likelihood:  -31.38
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

If the covariate 'x' is different for the two families, `x` and `xx`, say, then we only need to make the following changes

```
X = numeric(N)
X[1:n] = x
X[1:n + n] = NA

XX = numeric(N)
XX[1:n] = NA
XX[1:n + n] = xx

formula = Y  ~ X + XX -1
```

and add `XX` into the data.frame. Note how we can express the joint model as a 'union' of models with the use of `NA`'s to remove terms.

In the next example, we use also the `replicate` feature to estimate three replicated AR(1) models with the same hyperparamters, each observed differently.

```
## An example with three independent AR(1)'s with separate means, but
## with the same hyperparameters. These are observed with three
## different likelihoods.

n = 100
x1 = arima.sim(n=n, model=list(ar=c(0.9))) + 0
x2 = arima.sim(n=n, model=list(ar=c(0.9))) + 1
x3 = arima.sim(n=n, model=list(ar=c(0.9))) + 2

## Binomial observations
Nt = 10 + rpois(n,lambda=1)
y1 = rbinom(n, size=Nt, prob = exp(x1)/(1+exp(x1)))

## Poisson observations
Ep = runif(n, min=1, max=10)
y2 = rpois(n, lambda = Ep*exp(x2))

## Gaussian observations
y3 = rnorm(n, mean=x3, sd=0.1)

## stack these in a 3-column matrix with NA's where not observed
y = matrix(NA, 3*n, 3)
y[1:n, 1] = y1
y[n + 1:n, 2] = y2
```

```
y[2*n + 1:n, 3] = y3

## define the model
r = c(rep(1,n), rep(2,n), rep(3,n))
rf = as.factor(r)
i = rep(1:n, 3)
formula = y ~ f(i, model="ar1", replicate=r, constr=TRUE) + rf -1
data = list(y=y, i=i, r=r, rf=rf)

## parameters for the binomial and the poisson
Ntrials = rep(NA, 3*n)
Ntrials[1:n] = Nt
E = rep(NA, 3*n)
E[1:n + n] = Ep

result = inla(formula, family = c("binomial", "poisson", "normal"),
              data = data, Ntrials = Ntrials, E = E,
              control.family = list(
                      list(),
                      list(),
                      list()))
summary(result)
```

```
## Time used:
##     Pre = 0.107, Running = 0.263, Post = 0.0118, Total = 0.382
## Fixed effects:
##      mean    sd 0.025quant 0.5quant 0.975quant  mode   kld
## rf1 0.403 0.078      0.252    0.403      0.556 0.403 0.000
## rf2 0.975 0.054      0.864    0.976      1.076 0.976 0.000
## rf3 2.709 0.001      2.707    2.709      2.711 2.709 0.001
##
## Random effects:
##   Name     Model
##     i AR1 model
##
## Model hyperparameters:
##                                                mean       sd
## Precision for the Gaussian observations[3] 1.85e+04 1.83e+04
## Precision for i                            3.24e-01 6.10e-02
## Rho for i                                  8.38e-01 3.00e-02
##                                           0.025quant 0.5quant
## Precision for the Gaussian observations[3]   1248.791 1.28e+04
## Precision for i                                 0.215 3.21e-01
## Rho for i                                       0.775 8.39e-01
##                                           0.975quant     mode
## Precision for the Gaussian observations[3]   6.68e+04 3444.743
## Precision for i                              4.52e-01    0.318
## Rho for i                                    8.93e-01    0.840
##
## Marginal log-Likelihood:  -800.55
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

Models where the response/data depends on linear combinations of the "linear predictor" (or the sum of "fixed" and "random" effects)

In some cases, the data/response might depend on a linear combination of the "linear predictor" defined in the formula, like

```
y ~ 1 + z
```

then this implies that `y[1]` depends on `intercept + beta*z[1]`. Suppose if `y[1]` depends on `2*intercept + beta*z[1] + beta*z[2]`? Although it is possible to express this, using the tools we already have, it is more convenient to add another layer into the model. Let `A` be a `m x n` matrix, which defines new linear predictors, `eta~` from `eta`, like

```
eta~ = A %*% eta
```

Here, `eta` is the ordinary linear predictor defined using the formula, and the data depends on the linear predictor `eta~`. So we might express this as

```
y ~ 1 + z, with addition matrix A
```

meaning in short, that

```
y    ~ eta~   ## no intercept...
eta~ = A %*% eta
eta  = intercept + beta*z
```

This is specified by adding the A-matrix, using

```
control.predictor=list(A=A)
```

The argument `offset`, which might be defined in the formula as `offset(value)` or as an argument `inla(..., offset = value)`, does have different interpretation in the case where the A-matrix is used. The rule is that `offset` in the formula, goes into `eta`, whereas an argument `offset` goes into `eta~`. If we write out the expressions above adding offsets, `offset.formula` and `offset.arg`, we get

```
eta~ = A %*% eta + offset.arg
eta = intercept + beta*z + offset.formula
```

In the case where there is no A-matrix, then \verb|offset.total = offset.arg + offset.formula|.

The following example should provide more insight. You may change n and m, such that $m < n$, $m = n$ or $m > n$. Note that since the response has dimension m and the covariates dimension n, we need to use `list(y=y, z=z)` and not a `data.frame()`. This example also illustrates the use of offset's.

```
## 'm' is the number of observations of eta*, where eta* = A eta +
## offset.arg, and A is a fixed m x n matrix, and eta has length n. An
## offset in the formula goes into 'eta' whereas an offset in the
## argument of the inla-call, goes into eta*
n = 10
m = 100
offset.formula = 10+ 1:n
```

```
offset.arg = 1 + 1:m

## a covariate
z = runif(n)

## the linear predictor eta
eta = 1 + z + offset.formula

## the linear predictor eta* = A eta + offset.arg.
A = matrix(runif(n*m), m, n);
##A = inla.as.sparse(A)  ## sparse is ok
## need 'as.vector', as 'Eta' will be a sparseMatrix if 'A' is sparse
## even if ncol(Eta) = 1
Eta = as.vector(A %*% eta) + offset.arg

s = 1e-6
Y = Eta + rnorm(m, sd=s)

## for a check, we can use several offsets. here, m1=-1 and p1=1, so
## they m1+p1 = 0.
r = inla(Y ~ 1+z + offset(offset.formula) + offset(m1) + offset(p1),
        ## The A-matrix defined here
        control.predictor = list(A = A, compute=TRUE, precision = 1e6),
        ## we need to use a list() as the different lengths of Y
        ## and z
        data = list(Y=Y, z=z,
                m1 = rep(-1, n),
                p1 = rep(1, n),
                offset.formula = offset.formula,
                offset.arg = offset.arg),
        ## this is the offset defined in the argument of inla
        offset = offset.arg,
        ##
        control.family = list(initial = log(1/s^2), fixed=TRUE))
```

```
## Warning in inla.core(formula = formula, family = family, contrasts = contrasts, : The A-matrix in th
##   but an intercept is in the formula. This will likely result
##   in the intercept being applied multiple times in the model, and is likely
##   not what you want. See ?inla.stack for more information.
##   You can remove the intercept adding ``-1'' to the formula.
```

```
summary(r)
```

```
## Time used:
##     Pre = 0.0855, Running = 0.0951, Post = 0.00445, Total = 0.185
## Fixed effects:
##             mean    sd 0.025quant 0.5quant 0.975quant mode kld
## (Intercept)    1 0.001      0.999        1          1    1   0
## z              1 0.001      0.998        1          1    1   0
##
## Marginal log-Likelihood:  91203.32
##  is computed
```

```
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

```
## this should be a small number
print(max(abs(r$summary.linear.predictor$mean - c(Eta, eta))))
```

```
## [1] 3.29e-06
```

Here is a another example where the informal formula is

```
y = intercept + s[j] + 0.5*s[k] + noise
```

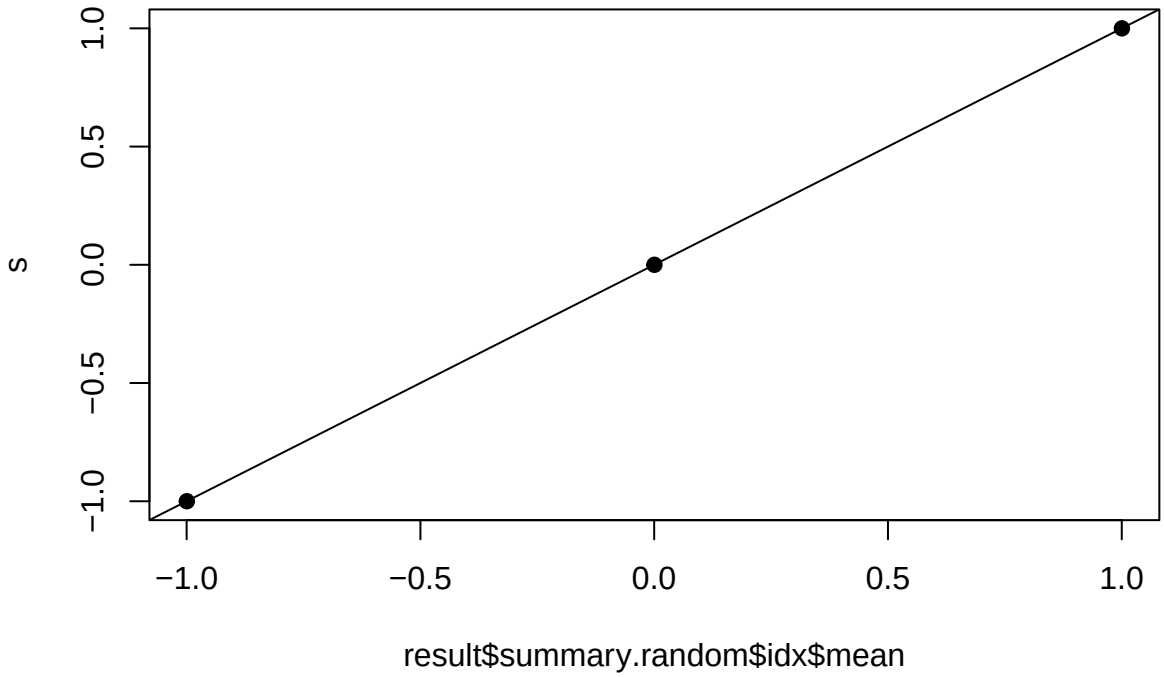
Instead of using the copy feature, we can implement this model using the A-matrix feature. What we do, is to first define a linear predictor being the intercept and s, then we use the A-matrix to 'construct the model'.

```
n = 100
s = c(-1, 0, 1)
nS = length(s)
j = sample(1L:nS, n, replace=TRUE)
k = j
k[j == 1L] = 2
k[j == 2L] = 3
k[k == 3L] = 1

noise = rnorm(n, sd=0.0001)
y = 1 + s[j] + 0.5*s[k] + noise

## build the formula such that the linear predictor is the intercept
## (index 1) and the 's' term (index 2:(n+1)). then kind of
## 'construct' the model using the A-matrix.
formula = y ~ -1 + intercept + f(idx)
A = matrix(0, n, nS+1L)
for(i in 1L:n) {
  A[i, 1L]        = 1
  A[i, 1L + j[i]] = 1
  A[i, 1L + k[i]] = 0.5
}

data = list(intercept = c(1, rep(NA, nS)), idx = c(NA, 1L:nS))
result = inla(formula, data=data, control.predictor=list(A=A))
## should be a straight line
plot(result$summary.random$idx$mean, s, pch=19)
abline(a=0,b=1)
```


s
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0
result$summary.random$idx$mean